

Sensitivity Analysis for Dummies

A cell-by-cell guide to the NSE Transform Sensitivity Notebook

A companion guide for water quality modellers at the workshop.

Introduction

Every model has inputs, knobs you can turn, dials you can set and parameters you can choose. Some of these knobs matter a lot. Turn them a little and the model's output swings wildly. Others barely matter at all; you could set them to almost any reasonable value and the model would barely notice.

Sensitivity analysis is the toolbox for answering the question: *which inputs matter?*

At its simplest, it means wiggling your parameters one at a time and watching what happens. But real models are messier than that. Parameters interact, the effect of turning knob **A** depends on where you've set knob **B**. And some parameters only matter when you look at certain parts of the output (low flows vs high flows, wet years vs dry years).

That's where **Shapley effects** come in.

Shapley Effects: Fair Attribution from Game Theory

Imagine 14 musicians playing in an orchestra. You want to know how much each musician contributes to the overall sound. You can't just mute them one at a time, the strings section sounds very different depending on whether the woodwinds are already playing. The contribution of the first violinist depends on who else is in the room.

Shapley values, invented by Lloyd Shapley (who won the 2012 Nobel Memorial Prize in Economic Sciences for this idea), solve exactly this problem. The approach is elegantly simple:

1. Consider every possible subset of musicians, from "no one playing" to "everyone playing."
2. For each musician, measure how much the sound changes when they join each subset.
3. Average across all possible orders and combinations.

The result is a **fair attribution**: each musician gets a score representing their average marginal contribution, properly accounting for every interaction. A musician who always improves the sound, regardless of who else is playing, gets a high Shapley value. A musician whose contribution depends on context gets a moderate score. A musician who never makes a difference gets zero.

In our world, the 14 parameters of the SAC-SMA model are the musicians, and the orchestra's "sound" is the model's NSE performance. **A Shapley effect is simply a Shapley value applied to model sensitivity** it tells you each parameter's fair share of the model's output variability.

Why This Matters

Most sensitivity analysis methods give you incomplete or misleading answers:

Method	What it does	The catch
One-at-a-time (OAT)	Change one parameter, hold others fixed	Misses all interactions. The answer depends on where you held the other parameters.
Sobol indices	Decomposes variance into main effects + interactions	Becomes explosively expensive with many parameters. Doesn't handle correlated parameters well.
Shapley effects	Game-theoretic fair attribution of each parameter's contribution	Computationally expensive, requires a surrogate model to be practical.

Shapley effects are the **gold standard** because they:

- Account for all interactions between parameters
- Work correctly even when parameters are correlated (something most other methods can't handle)
- Give you a single, interpretable number per parameter: "this parameter explains X% of the model's behaviour"
- Include **confidence intervals** so you know how much to trust the rankings

What This Notebook Does

This notebook applies Shapley effects to a 14-parameter rainfall-runoff model (SAC-SMA) for St Helens Creek, North Queensland. We compare two different ways of measuring model performance, sqrt-transformed NSE (standard practice) vs raw NSE and ask: *does the choice of performance metric change which parameters look important?*

Spoiler: yes, dramatically. The correlation between the two rankings is only 0.48.

What is Sensitivity Analysis and Why Should You Care?

The big picture question: When you run a rainfall-runoff model with 14 parameters, which ones actually matter?

You might think "all of them", but the answer is almost always "it depends." Sensitivity analysis is a set of tools that helps you answer:

- Which parameters drive the model's performance the most?
- Which parameters could you set to a default value and nobody would notice?
- How do your choices (e.g., which objective function you use) change the answer?

This notebook compares two versions of the Nash-Sutcliffe Efficiency (NSE):

- **Sqrt-NSE:** NSE calculated on the square root of discharge. This is the standard practice in hydrology because it stops the biggest floods from dominating your error metric.
- **Raw NSE:** NSE calculated on discharge directly. The big flows dominate, and the small flows (which might matter for water quality) get ignored.

The question is: Does this choice change which model parameters look important?

Cell-by-Cell Walkthrough

Title Cell (Markdown)

[![Open In Colab]](https://colab.research.google.com/...)

NSE Sensitivity: Square-Root Transformed vs Raw Discharge

What it is: The title and introduction. The Colab badge lets you click to open the notebook in Google's cloud environment (no installation needed on your machine).

Key concepts introduced:

- **Box-Cox transform ($\lambda=0.5$):** That's just a fancy name for taking the square root. It's a way to make data more "well-behaved", specifically, it makes the errors more evenly spread across low flows and high flows instead of being dominated by the few big days.
 - **Why 14 parameters:** The SAC-SMA model (Sacramento Soil Moisture Accounting) has 14 adjustable knobs. Some control how fast water moves through the soil, some control how much is held in storage, etc.
 - **St Helens Creek, North Queensland:** A real catchment in Australia. The data comes from here.
-

Cell 1 : Setup and Imports (Code)

```
IN_COLAB = 'google.colab' in sys.modules
```

```
if IN_COLAB:
```

```
    !pip install -q shapleyx
```

```
    # ... download data files
```

What it does: Checks whether you're running in Google Colab (the free cloud service) or locally on your own machine. If Colab, it installs the Python package shapleyx (which does the sensitivity analysis) and downloads the data files.

Then it imports the libraries:

- numpy : the workhorse of scientific Python; handles numbers and arrays
- pandas : for spreadsheets-in-Python (tables of data)
- matplotlib : for making graphs
- Sobol from scipy : generates special "well-spread-out" random numbers
- shapleyx : the star of the show; this package does the actual sensitivity analysis
- sacsma : the actual hydrological model code

What's happening conceptually: We're loading all the tools we need. Think of this as gathering your instruments before starting an experiment.

Section 1 : Load Forcing Data (Markdown + Code Cell 2)

```
df = pd.read_csv('data/st_helens_forcing_original.csv', parse_dates=['date'])
prcp = df['rainfall'].values
pet = df['pet'].values
Q_obs = df['Q_CUMEC'].values
```

What it does: Loads a CSV file (think: spreadsheet saved as text) containing:

- rainfall : daily rainfall (mm)
- pet : potential evapotranspiration (mm) (how much water the atmosphere could suck up)
- Q_CUMEC : observed streamflow (m³/s) at the catchment outlet
- Over 16,000 daily timesteps (about 46 years)
- A **warm-up** of 365 days is discarded : models need time to "spin up" their soil moisture stores from initial conditions

Why this matters: The quality of your sensitivity analysis depends on the data you feed it. Garbage in = garbage out. Here we have good, long observational records.

Section 2 : Parameter Definitions (Markdown + Code Cell 3)

```
param_names = [ 'UZW', 'UZFWM', 'LZW', 'LZFPM', 'LZFSM', 'ADIMP',  
                'UZK', 'LZPK', 'LZSK', 'ZPERC', 'REXP', 'PCTIM', 'PFREE', 'SIDE' ]
```

What it does: Defines the 14 parameters of SAC-SMA and their plausible ranges (lower bound lo, upper bound hi).

What these parameters mean (simplified):

Parameter	Name	What it roughly controls
UZW	Upper-zone tension water capacity	How much rain the top soil layer can hold before it runs off
UZFWM	Upper-zone free water capacity	How fast the top layer releases water sideways
LZW	Lower-zone tension water capacity	Deep soil moisture that plants can use
LZFPM	Lower-zone free water primary	Deep, slow groundwater that feeds baseflow
LZFSM	Lower-zone free water secondary	Faster-responding deep storage
ADIMP	Additional impervious area	Fraction of catchment that's effectively paved
UZK	Upper-zone depletion rate	How fast upper zone drains
LZPK	Lower-zone primary depletion rate	How fast slow baseflow drains
LZSK	Lower-zone secondary depletion rate	How fast fast baseflow drains
ZPERC	Maximum percolation rate	How fast water moves from upper to lower zone
REXP	Percolation shape exponent	Controls the nonlinearity of percolation
PCTIM	Permanent impervious area	Fraction of catchment that's always impervious
PFREE	Percolation free fraction	Fraction of percolation that goes directly to free water
SIDE	Deep recharge fraction	Water that escapes the catchment entirely

The big idea: We're going to wiggle these 14 parameters in a smart way and see which wiggles change the model output the most.

Section 3 : NSE Functions (Markdown + Code Cell 4)

```
def nse_sqrt(params):
    sim = mm_to_cumec(sacasma(prcp, pet, np.asarray(params, float))[WARMUP:])
    y_sim = sim ** 0.5
    ss_res = np.sum((Y_sqrt_obs - y_sim)**2)
    ss_tot = np.sum((Y_sqrt_obs - Y_sqrt_obs.mean())**2)
    return 1 - ss_res / ss_tot
```

What it does: Defines how we measure "how good is this parameter set?"

- Run SAC-SMA with the given parameters
- Convert from mm to m³/s using the catchment area
- Calculate NSE = 1 – SSE_residual / SSE_total
 - NSE = 1 means perfect fit
 - NSE = 0 means you're no better than guessing the mean flow
 - NSE < 0 means you're worse than just predicting the average

There are two versions:

1. **nse_sqrt:** Takes square root of both observed and simulated flow before calculating NSE
2. **nse_raw:** Uses raw flow values directly

Why the square root? Look at the flow range: 0 to 575 m³/s. A 10 m³/s error during a 500 m³/s flood counts the same as a 10 m³/s error during a 1 m³/s baseflow day. The sqrt-transform makes errors proportional rather than absolute, a 10× bigger flow gets about 3.2× ($\sqrt{10}$) more weight instead of 10× more weight.

Section 4 : Training Design: QMC Sobol' (Markdown + Code Cell 5)

```
N = 4096
X_u = Sobol(d=D, scramble=True, seed=42).random(n=N)
X = lo + X_u * (hi - lo)
```

What it does: Generates 4,096 different parameter sets to test.

Why Sobol? Imagine you have a 14-dimensional hypercube (that's the 14 parameters, each between its min and max). You need to pick 4,096 points inside this hypercube. If you use plain random numbers, you'll get clumps and gaps. **Sobol sequences** are "low-discrepancy", they spread points out more evenly, like the difference between throwing darts randomly vs placing them in a neat grid. This gives you better coverage of the parameter space with fewer model runs.

The key number: 4,096 = 2¹². Sobol sequences are designed to work well with powers of 2.

Section 5 : Evaluate SAC-SMA (Markdown + Code Cells 6 and 7)

```
for i in range(N):
    Y_sqrt[i] = nse_sqrt(X[i])
```

```
Y_raw[i] = nse_raw(X[i])
```

What it does: The expensive part. For each of the 4,096 parameter sets:

1. Run the full SAC-SMA simulation (16,000+ days)
2. Compute both NSE values
3. Store them

This takes about 13 seconds on a modern machine with 8 cores.

Cell 7 plots histograms showing the distribution of both NSE metrics across all 4,096 runs:

- Raw NSE has **4.7× more variance** than sqrt-NSE confirming that peak flows create much more scatter in model performance
- The mean NSE is lower for raw (0.69) than sqrt (0.78) , the big storms are harder to get right

Analogy: If you're a golfer, sqrt-NSE is like scoring based on how close to par you are. Raw NSE is like scoring where a triple-bogey on a par-5 counts way more than a birdie on a par-3. Same game, very different story.

Sections 6 & 7 : RS-HDMR Surrogate (Markdown + Code Cells 9-12)

```
surr_sqrt = rshdmr(df_s, polys=[10,6,2], method='omp_cv', n_iter=400, n_jobs=4, verbose=True)
```

What is a surrogate? We ran SAC-SMA 4,096 times. But to compute Shapley effects, we need to evaluate the model hundreds of thousands of times. Running SAC-SMA that many times would take days.

A **surrogate (or metamodel)** is a cheap, fast approximation of SAC-SMA. We train it on the 4,096 runs, then use it for the heavy computation. Think of it like: instead of cooking a full meal every time to taste-test a recipe, you learn the general rules of how ingredients affect taste, then predict the outcome without cooking.

RS-HDMR stands for "Random Sampling : High Dimensional Model Representation." It's a fancy regression method that:

1. Builds a set of mathematical "basis functions" (like building blocks)
2. Uses **OMP-CV** (Orthogonal Matching Pursuit with Cross-Validation) to pick only the most useful blocks
3. The result is a fast equation that approximates the model

How good are the surrogates?

- Sqrt-NSE surrogate: $R^2 = 0.971$ (explains 97.1% of the variance)
- Raw NSE surrogate: $R^2 = 0.989$

These are excellent , the surrogates are very faithful to the real model.

Cell 13 summarizes both surrogates side-by-side.

Section 9 : Vrugt Correlation Matrix (Code Cell 14)

```
C13 = np.array([...]) # 13×13 correlation matrix from Vrugt et al. (2006)
```

What it does: Loads a correlation matrix from the literature. SAC-SMA parameters aren't independent, if you increase one, another might need to change to keep the same behaviour. This matrix captures those relationships (e.g., UZTWM and LZTWM are strongly negatively correlated at -0.84).

Why include correlations? Parameters don't live in isolation. A good sensitivity analysis accounts for the fact that changing one parameter while holding others fixed might explore physically impossible combinations. By using a correlated joint distribution, we explore the parameter space more realistically.

The code then fixes any numerical issues (ensures the matrix is "positive definite", a technical requirement for the math to work).

Sections 10 & 11 : MC Shapley Effects (Code Cells 17 and 19)

```
joint = GaussianCopulaUniform(lows=lo, highs=hi, corr=corr)
```

```
eff_s, sh_s, var_s, lo_s, hi_s = shapley_effects(  
    f=surr_sqrt.predict, joint=joint,  
    N=2000, method='exhaustive', B=500, ...)
```

What is a Shapley effect? This is the key concept.

Imagine you have 14 players (parameters) contributing to a team score (NSE). You want to know how much each player contributes, accounting for the fact that they interact.

Shapley values come from game theory. The idea:

1. Try all possible "coalitions" of parameters (every subset from {UZTWM} to {all 14})
2. For each coalition, compute the average contribution of adding a particular parameter
3. Average over all possible orders and coalitions

The result: **Each parameter gets a score** representing its contribution to the model's output variance. Higher Shapley effect = more important.

Key parameters for `shapley_effects()`:

- `N=2000`: Number of Monte Carlo samples (more = more accurate)
- `method='exhaustive'`: Computes all possible coalitions (small enough with 14 parameters)
- `B=500`: Bootstrap samples for confidence intervals (500 resamples)
- `k_max=3`: Maximum interaction order to consider
- `alpha=0.05`: 95% confidence intervals

The output looks like this:


```

Param  Shapley    CI_lo    CI_hi
PFREE  0.25342  0.24167  0.26559
LZTWM  0.13035  0.12069  0.14171

```

...

How to read this:

- **PFREE** has a Shapley effect of 0.253, it contributes ~25% of the explained variance
- **CI_lo** and **CI_hi** are 95% confidence intervals, we're 95% sure the true value is between these. If the interval doesn't include zero, the parameter is "statistically significant."
- **ZPERC** has **CI_lo** = -0.00014 , it's barely significant (could be zero)

Why confidence intervals? We're doing Monte Carlo sampling, which introduces randomness. The confidence intervals tell us how much to trust the rankings. If two parameters have overlapping intervals, their order isn't certain.

These two cells do the same computation for sqrt-NSE and raw-NSE separately.

Section 12 : Side-by-Side Comparison (Code Cells 20-23)

```
comp['ΔRank'] = comp['Rk_sqrt'] - comp['Rk_raw']
```

...

Correlation(Sqrt, Raw Shapley): 0.483

What it does: Puts both sets of rankings next to each other and computes the differences.

The headline result: The correlation between sqrt and raw Shapley rankings is only **0.48**. That's like having two Top 40 music charts that barely agree, the choice of objective function completely reshuffles which parameters matter.

Key shifts:

Parameter	Sqrt rank	Raw rank	Shift
LZFPM	7	2	+5
REXP	10	5	+5
LZTWM	2	7	-5
ADIMP	5	10	-5

- **LZFPM** (deep groundwater storage) jumps from rank 7 to 2, it matters a lot when big flows are weighted because it controls baseflow volume after big storms
- **LZTWM** (soil moisture for plants) drops from rank 2 to 7, its errors (a few mm/day) are swamped by peak-flow errors (hundreds of m³/s) under raw NSE

The visualisations (bar charts, rank-shift plot, scatter plot) all tell the same story from different angles.

Section 13 : Interpretation (Markdown + Code Cell 24)

The punchline:

1. **The transform matters enormously.** Switching from sqrt-NSE to raw-NSE isn't a cosmetic change, it fundamentally reshapes which parameters appear important. The 0.48 correlation is low enough that you'd draw completely different conclusions about model structure.
2. **Use sqrt-NSE for calibration.** It's standard practice for good reason. It prevents the handful of biggest storm days from hijacking your calibration.
3. **If you must use raw NSE,** prioritise LZFSM, LZFPM, LZPK, PFREE, and REXP. These five dominate raw-NSE sensitivity.
4. **REXP is a sleeper hit.** Under sqrt-NSE it's negligible (rank 10, Shapley 0.019). Under raw-NSE it rockets to rank 5 (Shapley 0.101). If you're using raw NSE and you've been setting REXP to a default value – stop!
5. **Upper-zone parameters are consistently unimportant.** UZFWM, UZK, and PCTIM are always at the bottom. Don't waste calibration effort on them for this catchment.

Quick Reference: Key Concepts

Concept	In plain English
Sobol sequence	Smart way to pick sample points that covers the parameter space evenly
Surrogate model	A cheap approximation of the real model so we can run it millions of times
RS-HDMR	A regression method that builds the surrogate from building blocks
OMP-CV	An algorithm that picks only the most useful building blocks
R²	How well the surrogate matches the real model (1 = perfect)
Shapley effect	Each parameter's "fair share" of the model's output variance
Confidence interval	The range we're 95% sure the true value sits in
Bootstrap	Resampling with replacement to estimate uncertainty
Gaussian copula	A way to model correlated parameters mathematically
Box-Cox transform	Taking the square root to stabilise variance across flow ranges

How This Applies to Water Quality Modelling

The same logic applies to water quality models:

- Which water quality parameters (process rates, initial conditions, etc.) really drive your predictions?
- Does your choice of error metric (RMSE vs KGE vs NSE vs log-transform) change which parameters look important? Almost certainly yes.
- Which parameters can you fix with literature values vs which need site-specific calibration?

Sensitivity analysis isn't an abstract exercise, it tells you where to spend your calibration effort, what parameters to measure in the field, and how much confidence to have in your predictions.

This guide was written for the NSE Transform Sensitivity workshop. The companion notebook is at [sac_sma_nse_transform/nse_transform_sensitivity.ipynb](#).